How a bouquet upload is saved

Student file: Acuesta_John_Constantine_Finals_LabActivity1_HOW_IT_WORKS.pdf
Theme: Petal Express (Laravel 11 + Blade + Breeze, SQLite).

This PDF walks one photo from the Add bouquet button to the folder.
It names the watermark file, the saved WebP, and the database column.
It also says why each name is shaped that way.

Intervention does not write a separate thumbnail.
The small admin photo is the same WebP, drawn small in the browser.

Longer notes: docs/FINALS-LAB-ACT-1-CODE.md
Demo login: admin@test.com / 123

There are two picture files. public/images/watermark.png is the shared stamp. Its name stays watermark.png because AdminPostController always opens that path. storage/app/public/products/<uuid>.webp is the bouquet photo Intervention just encoded. A new UUID is used so a second upload cannot overwrite the first. The original JPG or PNG name is thrown away.

```
Two picture files

Stamp (one shared file, committed with the app)
  petal-express/public/images/watermark.png
  PNG, 560 x 160, transparent background

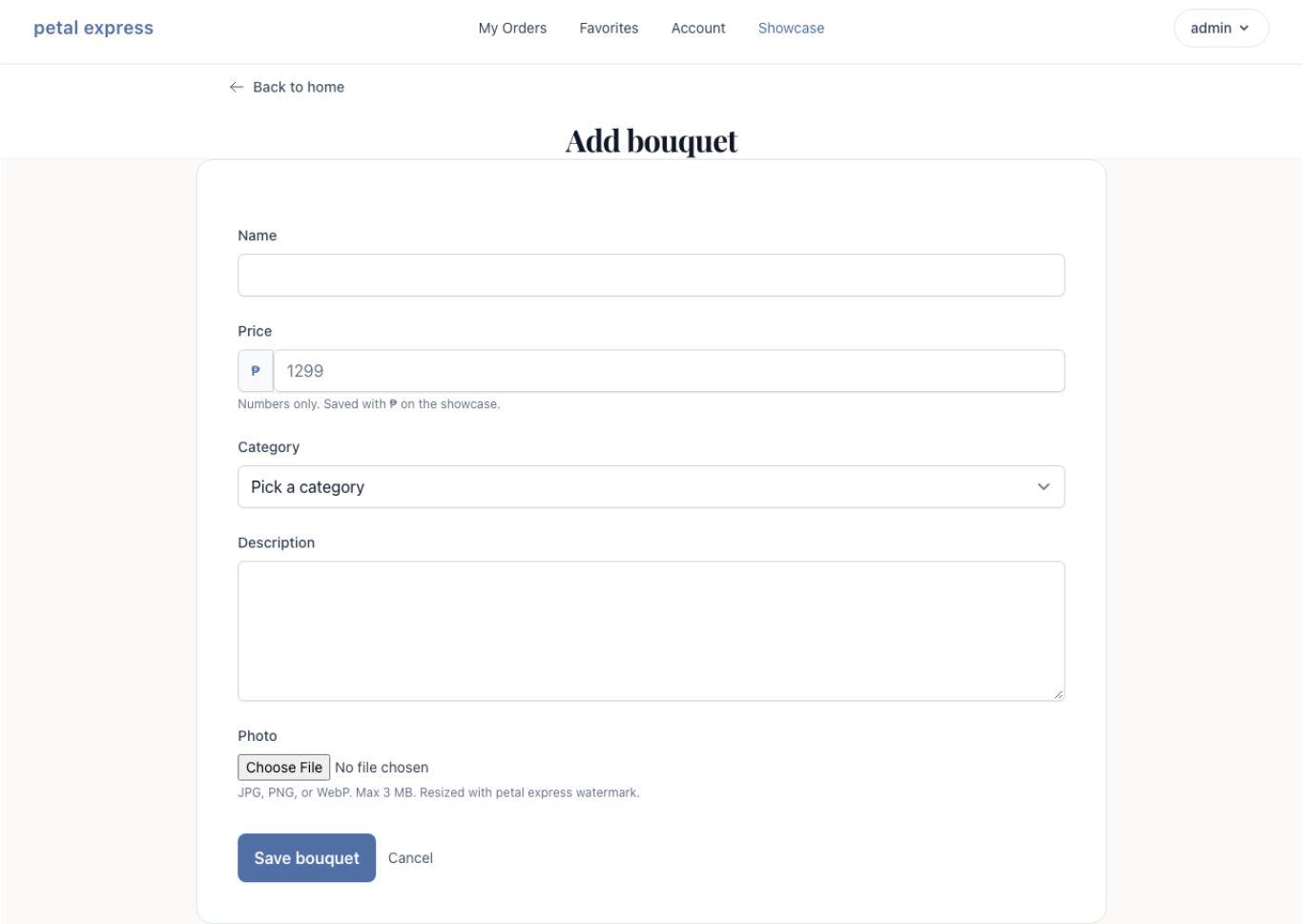
Saved bouquet photo (new file on every upload)
  petal-express/storage/app/public/products/<uuid>.webp
  example: products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp
  file(1) says: WebP image, 800 x 800

These are not the same file.
The stamp is not copied into products/.
The upload's original name is not kept.
```

This is the actual stamp file, public/images/watermark.png. It is a PNG with a transparent background, 560 by 160 pixels on this machine. It is part of the project, not an upload. saveWatermarkedImage() scales it to at most 38 percent of the photo width, and never wider than 320 pixels, then paints it 12 pixels in from the bottom-right at 55 percent opacity.



The Add bouquet form posts to admin.posts.store. The Blade file is resources/views/admin/posts/create.blade.php. enctype="multipart/form-data" is required or PHP never receives the file. The input name is image, which is the key PostRequest validates. @csrf is Laravel's check that the form came from our own site.



PostRequest runs before the controller saves anything. On create, image is required. Allowed types are JPG, PNG, and WebP. max:3072 means 3 MB. On edit the rule is nullable, so a blank file input keeps the old photo. A failed rule redirects back to the form. saveWatermarkedImage() does not run, so no WebP is written.

Step 4 — PostRequest image validation

```
// PostRequest.php
  21 |     public function rules(): array
  22 |     {
  23 |         $imageRules = $this->isMethod('put') || $this->isMethod('patch')
  24 |             ? ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:3072']
  25 |             : ['required', 'image', 'mimes:jpg,jpeg,png,webp', 'max:3072'];
  26 |
  27 |         return [
  28 |             'title' => ['required', 'string', 'max:255'],
  29 |             'body' => ['required', 'string'],
  30 |             'price' => ['required', 'integer', 'min:0'],
  31 |             'category_id' => ['required', 'integer', 'exists:categories,id'],
  32 |             'image' => $imageRules,
  33 |         ];
  34 |     }
```

saveWatermarkedImage() is private on AdminPostController. Image::decode() reads the upload so JPG and PNG become the same in-memory picture. scaleDown(width: 1000) shrinks only photos wider than 1000 pixels. insert() stamps the watermark. encodeUsingFormat(Format::WEBP, quality: 80) makes the bytes. Storage::disk('public')->put() writes them. The method returns the relative path. store() saves that string in showcase_flowers.image.

Step 5 — saveWatermarkedImage() core

```
// AdminPostController.php
 148 |     // resize, watermark, save as webp under storage/app/public/products
 149 |     private function saveWatermarkedImage(UploadedFile $file): string
 150 |     {
 151 |         $image = Image::decode($file);
 152 |         $image->scaleDown(width: 1000);
 153 |
 154 |         $watermarkPath = public_path('images/watermark.png');
 155 |         if (! is_file($watermarkPath)) {
 156 |             throw new \RuntimeException('Watermark not found: public/images/watermark.png');
 157 |         }
 158 |
 159 |         $watermark = Image::decode($watermarkPath);
 160 |         // bigger watermark: up to 38% of photo width, capped at 320px
 161 |         $watermarkWidth = min(
 162 |             320,
 163 |             max(1, (int) ($image->width() * 0.38))
 164 |         );
 165 |         $watermark->scaleDown(width: $watermarkWidth);
 166 |
 167 |         $image->insert(
 168 |             $watermark,
 169 |             12,
 170 |             12,
 171 |             Alignment::BOTTOM_RIGHT,
 172 |             0.55
 173 |         );
 174 |
 175 |         $path = 'products/'.Str::uuid().'.webp';
 176 |         $encoded = $image->encodeUsingFormat(
 177 |             Format::WEBP,
 178 |             quality: 80
 179 |         );
```

The column does not store pixels. It stores products/ plus a UUID plus .webp. products/ keeps bouquet photos in one folder on the public disk. Str::uuid() runs before Post::create(), so the name cannot be the new row id yet. .webp matches Format::WEBP. The visitor filename is not used, because rose.jpg can be uploaded twice and can contain spaces.

Why the saved name looks like that

```
Stored in showcase_flowers.image:
  products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp

products/     folder on the public disk
0f95b48d-2d8d-4a6e-8406-0aa6522e34ec
              Str::uuid()  (8-4-4-4-12 hex)
.webp         Format::WEBP, quality 80

Not used: the visitor filename (rose.jpg)
Not used: the database id
Why: two uploads must not overwrite each other,
and the file is written before the new id exists.

Browser URL:
  /storage/products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp
```

The public disk root is storage/app/public, from config/filesystems.php. Browsers cannot read that folder until php artisan storage:link creates public/storage as a shortcut. The URL is /storage/ plus the column value. Row 107 stores products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp for Manual PDF PNG Upload. displayImageUrl() returns /storage/products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp.

Symlink and the database path

```
$ ls -ld public/storage
public/storage -> storage/app/public

Disk root:  storage/app/public
Web prefix: /storage
Shortcut:   public/storage   (php artisan storage:link)

sqlite> SELECT id, title, image FROM showcase_flowers
        WHERE id = 107;
107 | Manual PDF PNG Upload
    | products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp

The column stores the path. It does not store pixels.
```

This figure is that saved file itself, not a screenshot of the page. The file command reports WebP, 800 by 800. Width stayed 800 because scaleDown only shrinks photos wider than 1000. The mark is in the bottom-right. This is the only image file Intervention wrote for that upload.



There is no thumbnail file. The admin list, the edit page, the preview modal, and the public detail page all use Post::displayImageUrl(). CSS makes the admin box small. The modal and the detail page show the same URL larger, which is why you can see the watermark there. One file means force delete has one path to remove.

```
The thumbnail is the same WebP drawn small

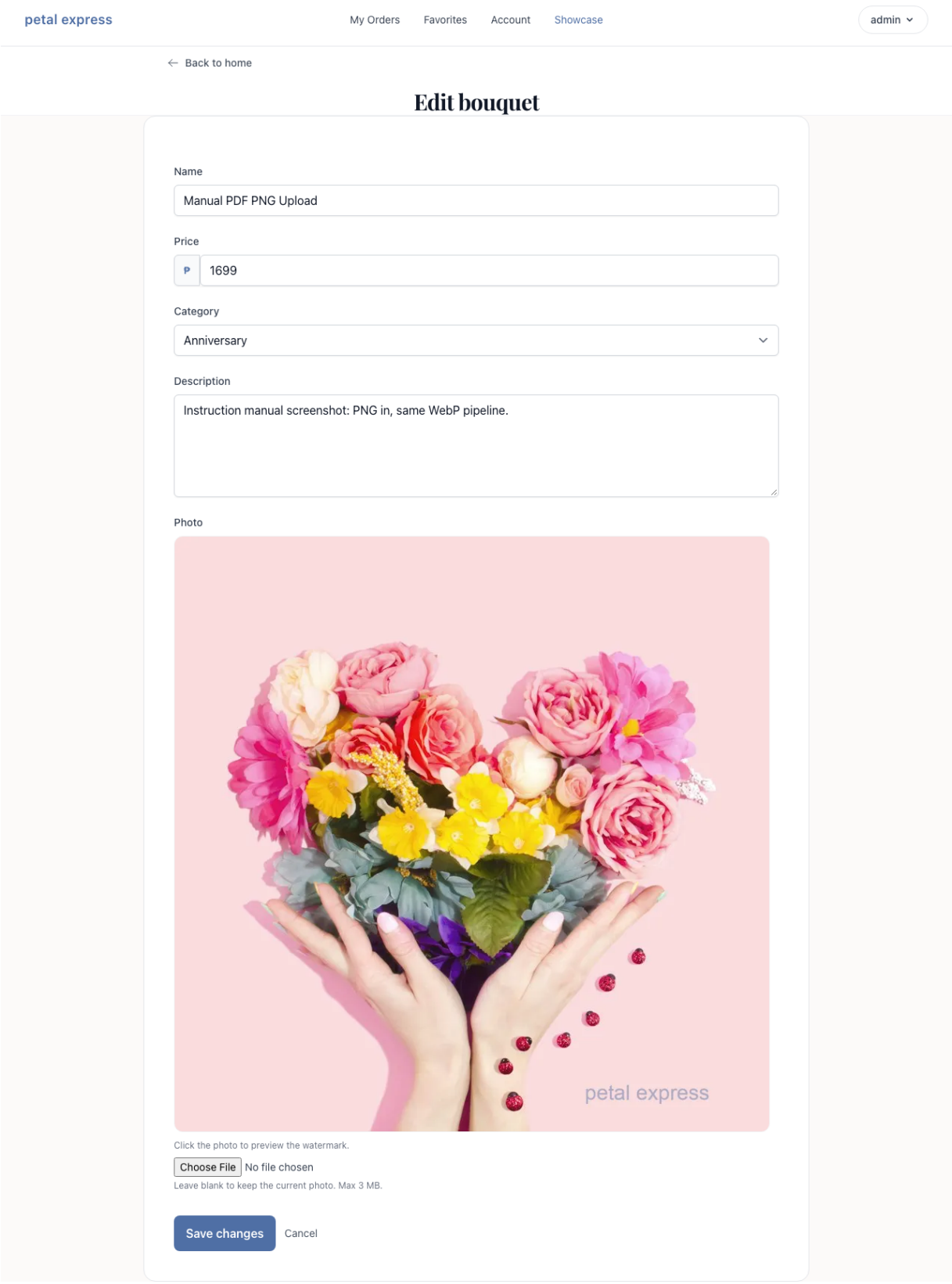
 Intervention writes ONE file:
   storage/app/public/products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp

 There is no products/<uuid>-thumb.webp.

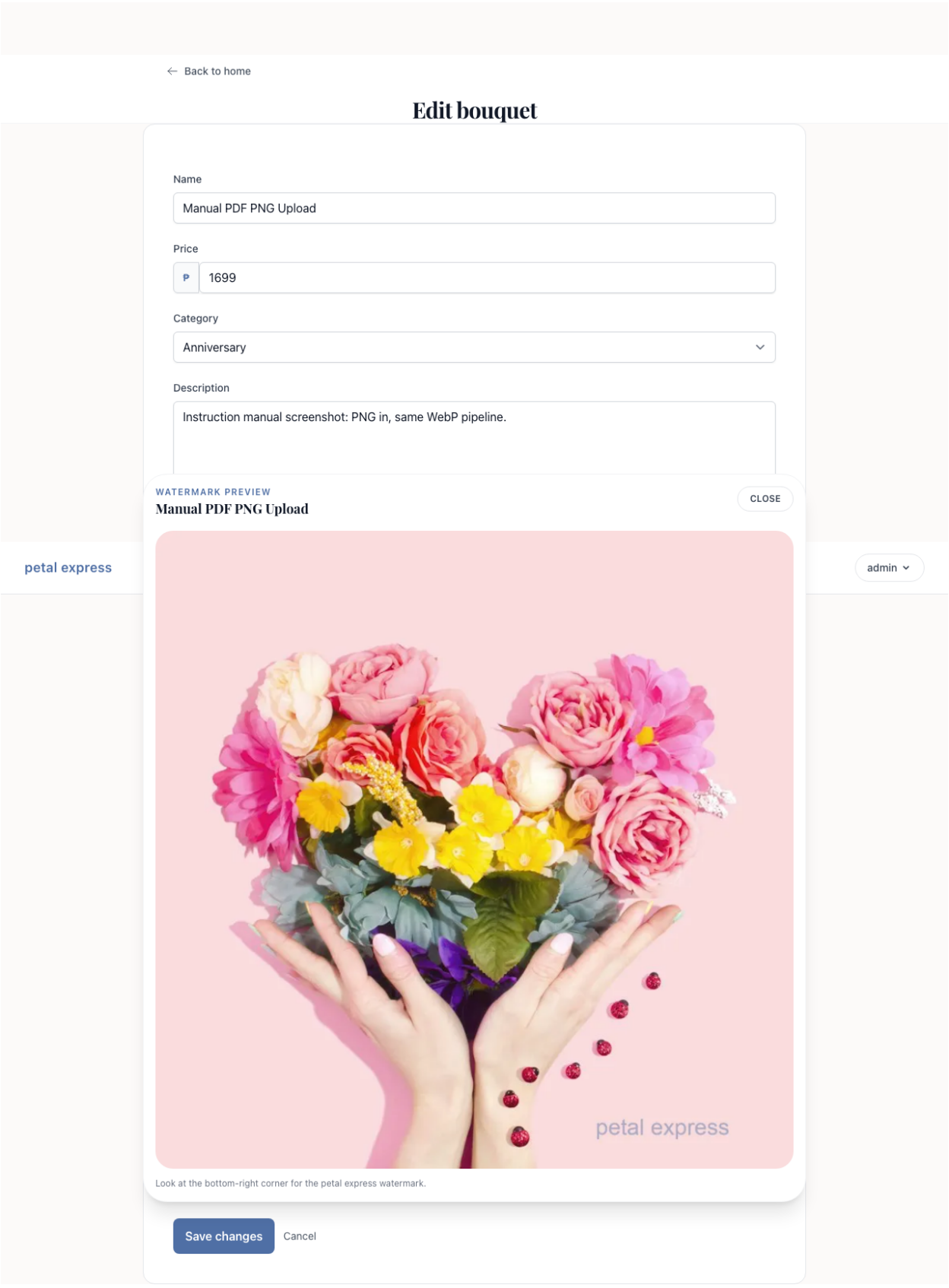
 Admin list box:  about 80px, same URL
 Edit thumb:      same URL
 Preview modal:   same URL, drawn larger
 Public detail:   same URL, min-height 420px

 Method they all call:
   Post::displayImageUrl()
   return '/storage/' + image column
```

The edit screen looks like a thumbnail. The img src is still /storage/products/<uuid>.webp. admin-photo-thumb only changes the box size. Click it and Alpine opens the preview with that same src. Seeded bouquets with an empty image column do not use this file. They fall back to public/images/bouquets/01.jpg through 06.jpg, which are not watermarked.



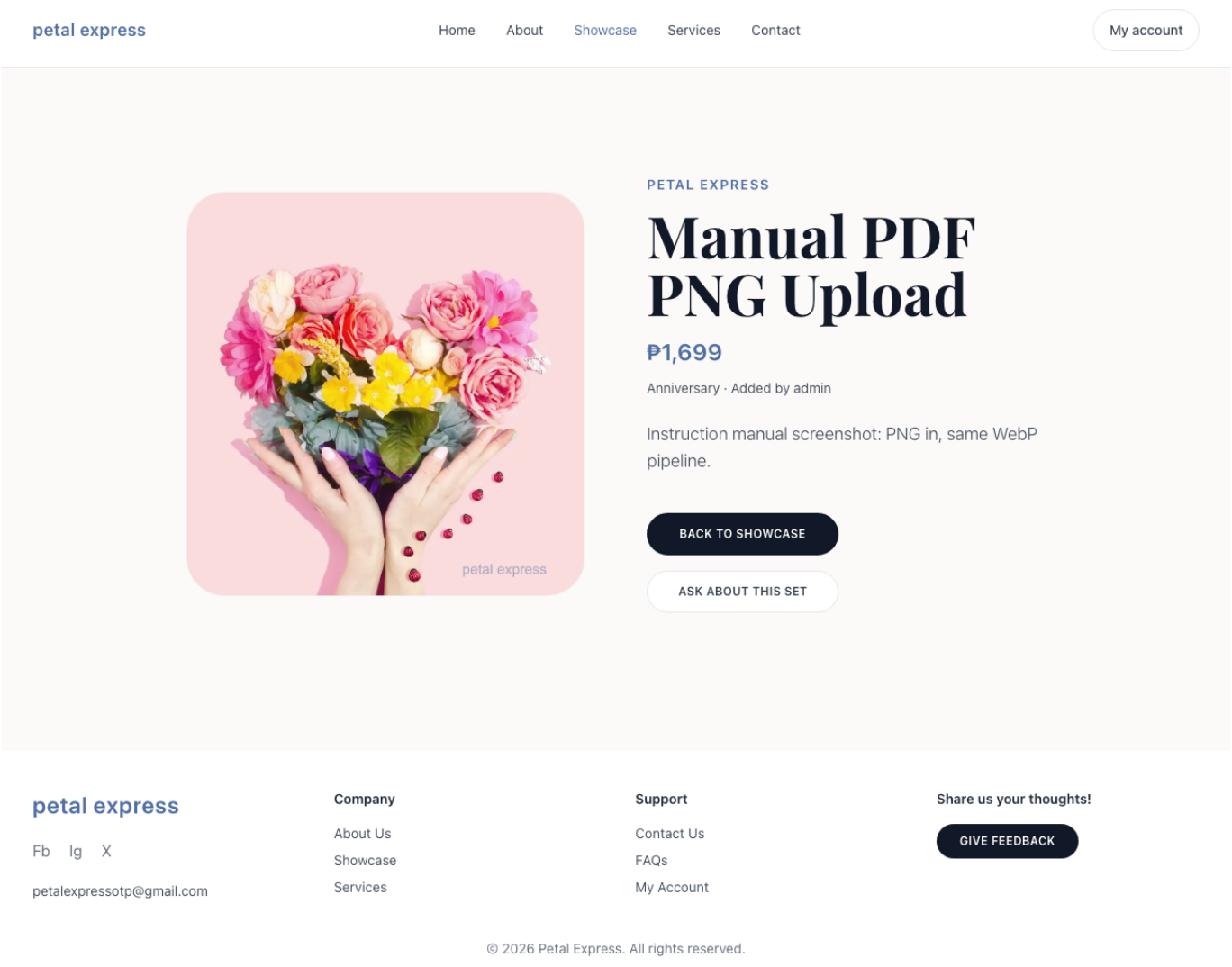
The preview modal loads the same WebP. It does not ask Intervention to render again. Intervention already burned the mark into the file at save time. The helper text tells you to look at the bottom-right corner.



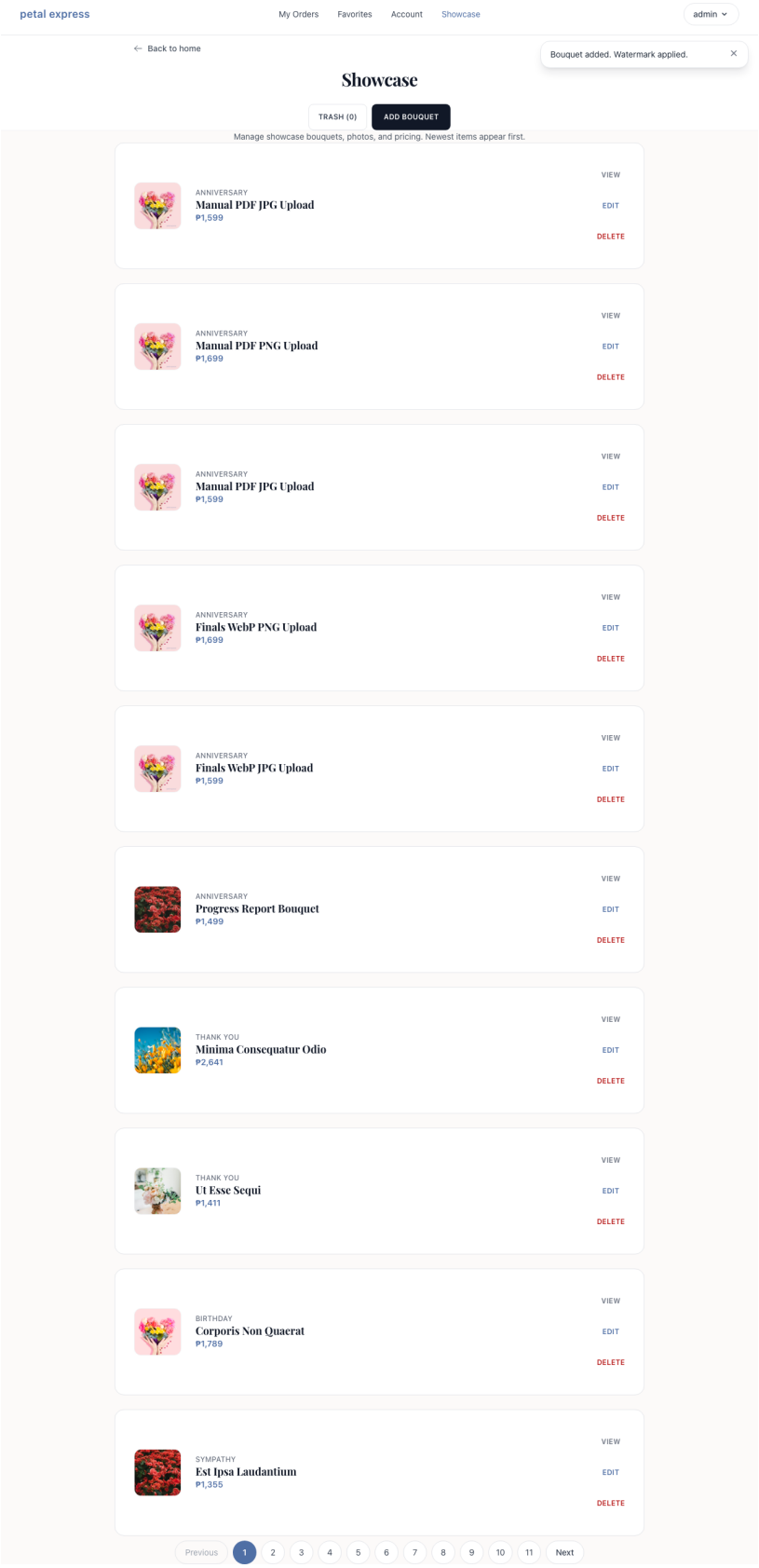
Close-up of the saved WebP. The mark is the contents of watermark.png after scaleDown and insert(). Offset 12, 12 is the gap from the bottom-right corner. 0.55 keeps 55 percent of the mark's opacity. The GD driver in Intervention treats 1 as full strength.



The public detail page calls displayImageUrl() and puts it in an img tag. The page does not decode, resize, or watermark. If the photo looks unmarked, you opened a seeded row whose image column is empty. Open a bouquet you uploaded, such as Manual PDF PNG Upload.



After a successful create, store() redirects to the admin list. The flash text is Bouquet added. Watermark applied. That text is the signal that saveWatermarkedImage() returned a path and Post::create() wrote the row. The original JPG is not in storage/app/public/products. Only the WebP is.



Delete on the list calls destroy(), which sets deleted_at. The WebP stays in storage/app/public/products, so Trash can still show it. Delete forever calls forceDelete(), which removes the row and then Storage::disk('public')->delete() on that products/<uuid>.webp path. Restore does not move a file, because the file never left.

